Software Design Specification

SwishVision: Basketball Analytics Platform

Mallick Mikaal Imam Minhaj ul Hassan Muhammad Ahmed Silat

Syed Muhammad Sameer Hassan

Version 1.10
11th May 2026

Project Team	
Mallick Mikaal Imam	
Minhaj ul Hassan	
Muhammad Ahmed Silat	
Syed Muhammad Sameer Hassan	

Submission Date	11th May 2026

Document History

Version	Name	Date	Description of Change
1.00	Ahmed Silat	16 Mar 2026	Initial draft, architecture and design documented
1.10	Minhaj ul Hassan	11 May 2026	Post-implementation update, finalised technology stack table, replaced PostgreSQL with SQL Server (pyodbc), updated file system layout to match shipped `app/` structure, updated environment variables
1.11	Minhaj ul Hassan	11 May 2026	Corrected output codec across the document, pipeline writes H.264 `.mp4` (via `mp4v` then `imageio-ffmpeg` transcode). Updated output filename pattern to actual `output_session_{id}_{n}_processed.mp4`.

Document Information

Project	SwishVision
Document	Software Design Specification
Document Version	1.10
Status	Final
Author(s)	Mallick Mikaal Imam, Minhaj ul Hassan, Muhammad Ahmed Silat, Syed Muhammad Sameer Hassan
Approver(s)	Course Supervisor

Contents

1 Introduction

1.1 Purpose of Document

This document provides the design specifications for SwishVision. It outlines both the high-level architectural view and the detailed low-level design of the system, covering the web platform, the CV processing pipeline, data models, and their interactions. The document serves as the blueprint that guides implementation and ensures alignment between design decisions and the requirements stated in the SRS.

1.2 Intended Audience

This document is intended for the development team building the system, the course supervisor evaluating the design, and any future maintainers.

1.3 Document Convention

- Font: Arial (Latin Modern as LaTeX substitute)
- Size: 11
- Line Spacing: 1.5

1.4 Project Overview

SwishVision is a web-based analytics platform that transforms basketball practice footage into structured performance data using computer vision and pose estimation. A fully working CLI prototype already exists, built on YOLOv8 for object detection and pose estimation. The object detection model (`best.pt`) was fine-tuned from YOLOv8 over 13 training iterations on the Basketball-1 Roboflow dataset. Human pose estimation uses `yolov8m-pose.pt`, tracking 17 body keypoints.

The design objective of this phase is to wrap that pipeline in a web application, adding user management, video upload, asynchronous processing, and a results dashboard.

The pipeline executes 12 steps: (1) frame extraction, (2) ball detection (`BallTracker`), (3) rim detection (`RimTracker`), (4) pose detection (`HumanTracker`), (5) track validation, (6) trajectory interpolation, (7) joint angle calculation, (8) ball-hand interaction (`ball_hand()`), (9) shot start identification (`shot_started()`), (10) frame annotation, (11) shot outcome detection (`ShotTracker`), and (12) output video write and report generation.

1.5 Scope

- Design of the web application layer (frontend + backend API)
- Design of the user authentication subsystem

- Design of the video upload and storage subsystem
- Design of the asynchronous job queue for video processing
- Design of the database schema
- Integration design between the web layer and the existing CLI pipeline
- Design of the results dashboard and reporting views

2 Design Considerations

2.1 Assumptions and Dependencies

- Existing pipeline modules (`BallTracker`, `RimTracker`, `HumanTracker`, `ShotTracker`, drawers) are stable internal libraries, invoked by a worker, not modified.
- Pre-trained model files are available under `models/`: `best.pt` (∼43 MB) and `yolov8m-pose.pt` (∼78 MB).
- Additional model variants (`bestOld.pt`, `bestYT.pt`, `ballRim.pt`) exist but are not used in the primary pipeline.
- Video storage is local filesystem-based for the current version.
- A Microsoft SQL Server 2019+ instance is available with the appropriate ODBC driver.
- The task queue uses Celery with Redis as broker.
- Output videos are written in two stages: `mp4v` via OpenCV, then transcoded to H.264 (`libx264`, `yuv420p`, `+faststart`) using `imageio-ffmpeg`.
- Intermediate data files (`angs.txt`, `xy_coords.txt`, `detections.txt`, `ball_locl.txt`) are transient.
- `stubs_utils.py` can cache and reload intermediate tracking results.

2.2 Risks and Volatile Areas

- **Model output variability:** Accuracy varies with video quality, lighting, and angle.
- **Processing time unpredictability:** Inference time varies; the async queue decouples upload from results.
- **File storage growth:** Videos and outputs can be large; retention policy required.
- **Interpolation artefacts:** `interpolate_missing_tracks()` can produce false positives.
- **Shot start detection sensitivity:** `shot_started()` depends on pose keypoints and ball-hand distance.
- **Multiple model variants:** Web layer must use `best.pt` as the canonical production model.

3 System Architecture

3.1 System Level Architecture

SwishVision follows a three-tier architecture: a client browser communicates with the Web Application Server over HTTPS; processing is offloaded to a Background Worker via a task queue; the worker writes results to the database and filesystem.

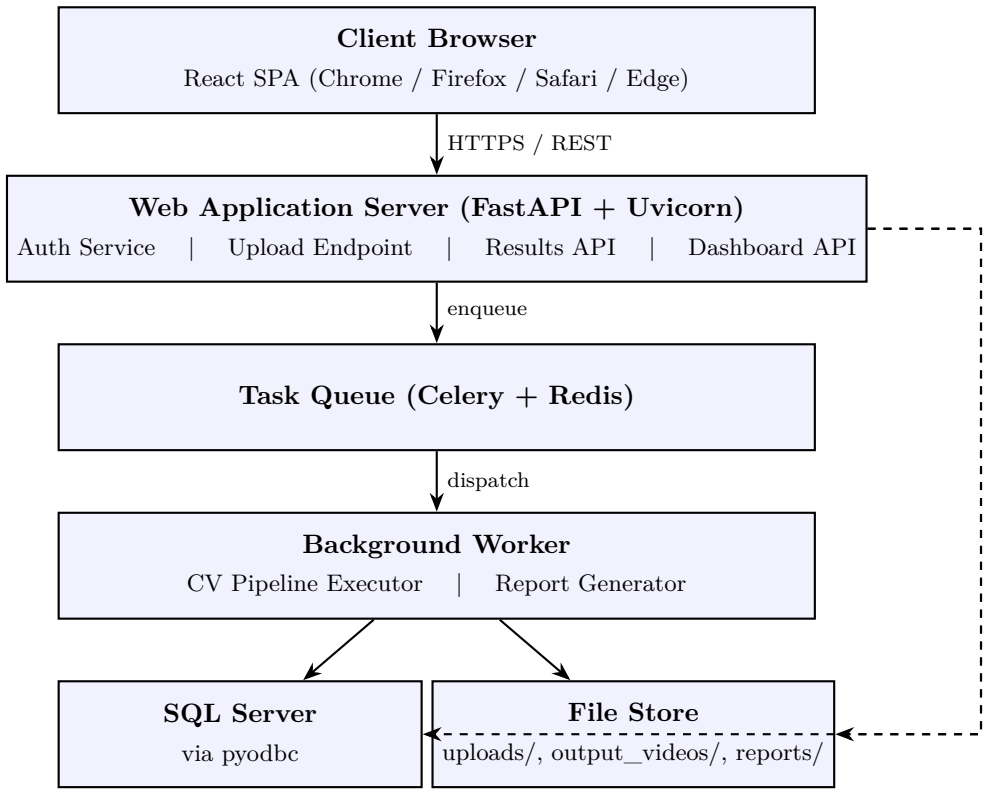

Figure 1: System Architecture

3.2 Software Architecture

3.2.1 Module Decomposition

1. Auth Module: registration, login, session management. Exposes `POST /api/register`, `POST /api/login`, `POST /api/logout`.

2. Video Module: upload, storage, lifecycle. Exposes `POST /api/sessions/upload`, `GET /api/sessions`, `GET /api/sessions/{id}`, `DELETE /api/sessions/{id}`.

3. Processing Module (Worker): Celery task that invokes the CV pipeline.

4. CV Pipeline (existing): trackers, drawers, utils. No direct web exposure.

5. Report Module: structures pipeline output for API responses. Exposes `GET /api/sessions/{id}/report`, `GET /api/sessions/{id}/shots`.

6. Dashboard Module: aggregates session data. Exposes `GET /api/dashboard/summary`,

`GET /api/dashboard/trends`.

7. Frontend: React UI consuming the REST API. Key views: Landing/Login, Registration, Dashboard, Upload, Session Results, History.

3.2.2 Technology Stack

Layer	Technology
Frontend	React 18 (Create React App), JavaScript, Node.js 18+ dev server
Backend Framework	FastAPI 0.135 on Uvicorn 0.42 (ASGI)
ORM / Migrations	SQLAlchemy 2.0 + Alembic 1.14
Task Queue	Celery 5.6 (worker with `-pool=solo` on Windows)
Message Broker	Redis 7+ (`redis://localhost:6379/0`)
Database	Microsoft SQL Server 2019+ via `pyodbc` (`mssql+pyodbc`, ODBC Driver 17)
CV Pipeline	Python 3.8+, PyTorch $\geq$2.0, Ultralytics YOLOv8 $\geq$8.0, OpenCV $\geq$4.8
Authentication	JWT (HS256) via `python-jose`; passwords hashed with `bcrypt` (`passlib`)
Video Storage	Local filesystem: `app/uploads/` (raw), `output_videos/` (annotated H.264 `.mp4`); reports to `reports/session_{id}_{n}_report.txt`

4 Design Strategy

The core design strategy is **wrapper-first**: the CV pipeline is treated as a stable black box, and the web application is built around it rather than integrating deeply with its internals. This minimizes regression risk to the existing working system.

Asynchronous by default: Video processing is never done synchronously in a request-response cycle. Every upload enqueues a job; the client polls or receives a notification when complete.

Separation of concerns: The web API, the background worker, and the CV pipeline are isolated processes.

Database as source of truth for results: Pipeline output is parsed from the text report and structured video data, then stored relationally.

Progressive enhancement: Core functionality (upload, view results) works on basic clients; a richer experience is provided where JavaScript is available.

5 Detailed System Design

5.1 Database Design

5.1.1 ER Diagram

Figure 2: ER Diagram

5.1.2 Data Dictionary

Table: `users`

Column	Type	Constraints	Description
`user_id`	UUID / SERIAL	PRIMARY KEY	Unique user identifier

email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
display_name	VARCHAR(100)		Display name
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Creation timestamp

Table: sessions

Column	Type	Constraints	Description
session_id	UUID / SERIAL	PRIMARY KEY	Unique session ID
user_id	UUID / INT	FK → users, NOT NULL	Owner
video_filename	VARCHAR(500)	NOT NULL	Original filename
video_path	VARCHAR(1000)	NOT NULL	Server path to raw video
output_video_path	VARCHAR(1000)		Path to annotated output
status	VARCHAR(50)	NOT NULL, DEFAULT 'queued'	queued/processing/completed/failed
upload_time	TIMESTAMP	NOT NULL, DEFAULT NOW()	Upload timestamp
completed_time	TIMESTAMP		Processing finish time
error_message	TEXT		Error detail if failed
fps	FLOAT		FPS of source video

Table: reports

Column	Type	Constraints	Description
report_id	UUID / SERIAL	PRIMARY KEY	Unique report ID
session_id	UUID / INT	FK → sessions, UNIQUE	Parent session
total_shots	INT	NOT NULL	Detected shot attempts
made_shots	INT	NOT NULL	Made shots
shot_percentage	FLOAT		Made / total × 100

consistency_score	FLOAT		Form consistency (0–100)
avg_release_angle	FLOAT		Average release angle (deg)
feedback_text	TEXT		Plain-language form feedback
report_file_path	VARCHAR(1000)		Path to raw .txt report

Table: shot_events

Column	Type	Constraints	Description
shot_id	UUID / SERIAL	PRIMARY KEY	Unique shot ID
session_id	UUID / INT	FK → sessions, NOT NULL	Parent session
shot_index	INT	NOT NULL	Ordinal index (1-based)
start_frame	INT		Frame shot began
release_frame	INT		Frame ball left hand
outcome	VARCHAR(10)		"made" or "missed"
release_angle	FLOAT		Release angle (deg)
elbow_angle_at_release	FLOAT		Elbow angle at release
wrist_angle_at_release	FLOAT		Wrist angle at release

Table: angle_frames

Column	Type	Constraints	Description
angle_frame_id	UUID / SERIAL	PRIMARY KEY	Unique record ID
shot_id	UUID / INT	FK → shot_events, NOT NULL	Parent shot
frame_index	INT	NOT NULL	Video frame number
elbow_angle	FLOAT		Elbow angle (deg)
wrist_angle	FLOAT		Wrist angle (deg)
knee_angle	FLOAT		Knee angle (deg)
hip_angle	FLOAT		Hip angle (deg)

5.2 Application Design

5.2.1 Class Diagram

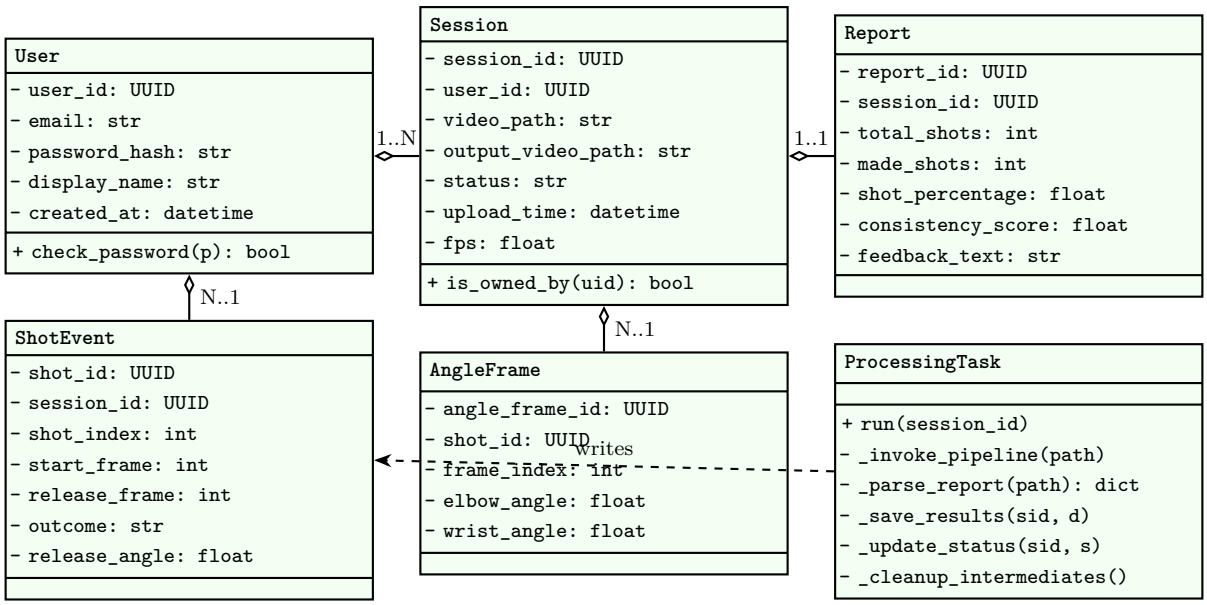

Figure 3: Class Diagram (Web Layer)

Existing pipeline classes (reused unchanged): `BallTracker`, `RimTracker`, `HumanTracker`, `ShotTracker`, `BallTracksDrawer`, `RimTracksDrawer`, `HumanTracksDrawer`, and the utilities `read_video`, `write_video`, `ball_hand`, `shot_started`.

5.2.2 Sequence Diagrams

Sequence 1: Video Upload and Processing

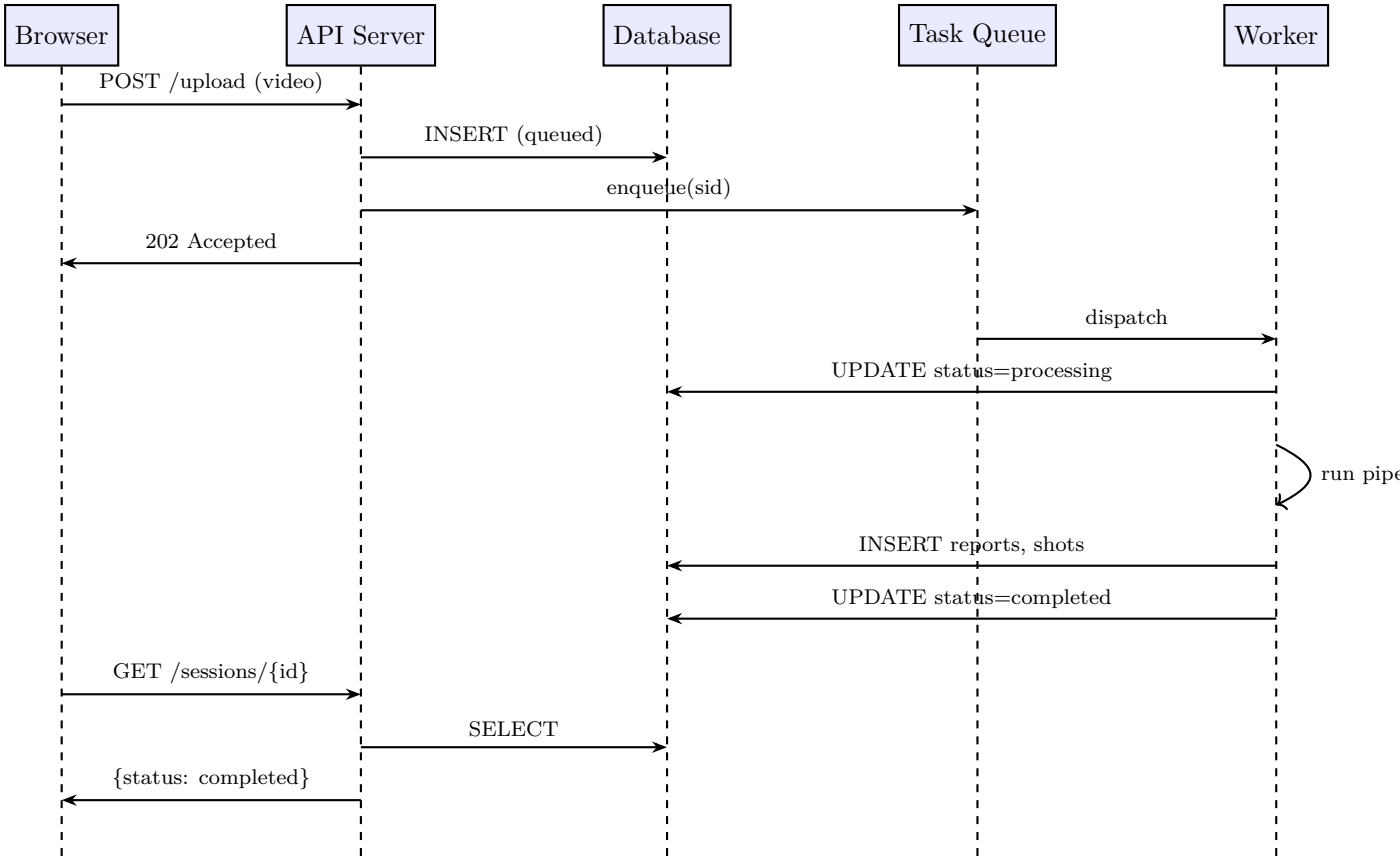

Figure 4: Sequence Diagram: Upload and Processing

Sequence 2: View Shot Analytics

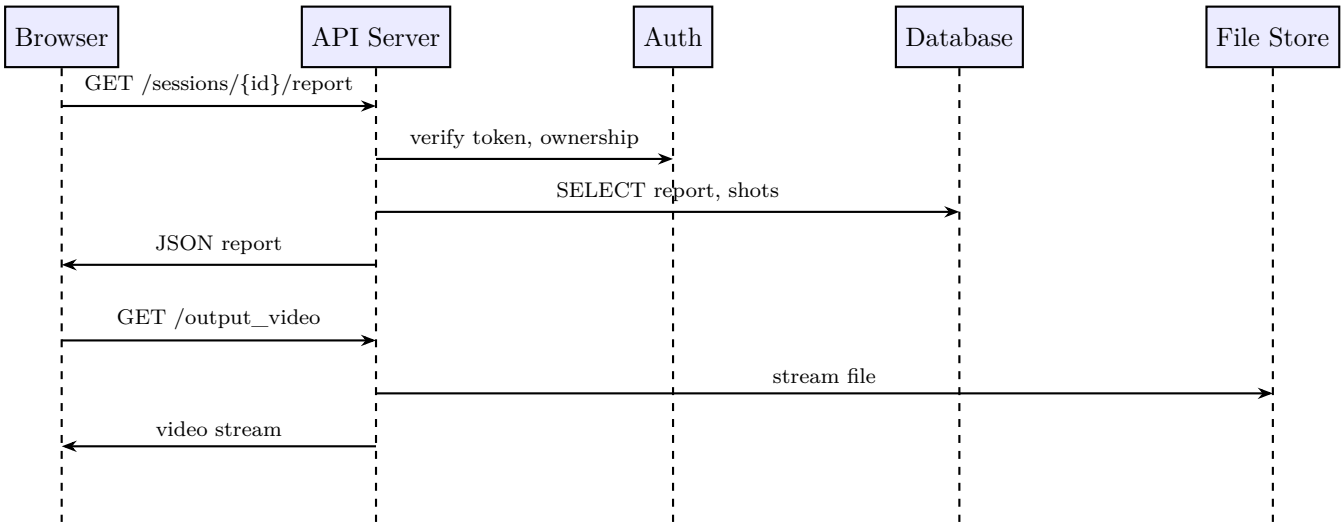

Figure 5: Sequence Diagram: View Shot Analytics

5.2.3 State Diagrams

Session State Machine

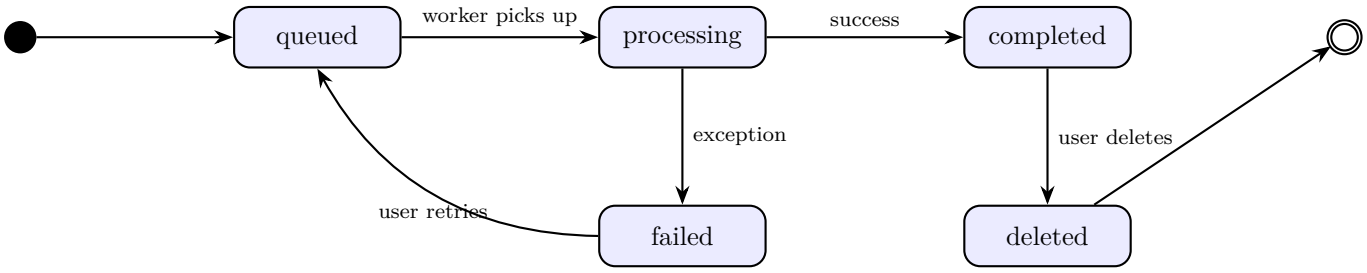

Figure 6: Session State Diagram

User Authentication State

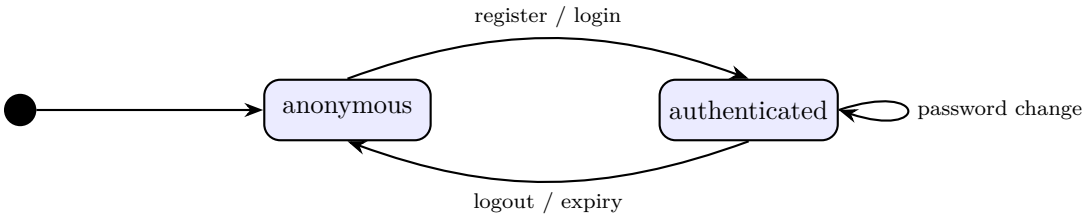

Figure 7: Authentication State Diagram

5.2.4 Activity Diagrams

Full Pipeline Execution (Worker)

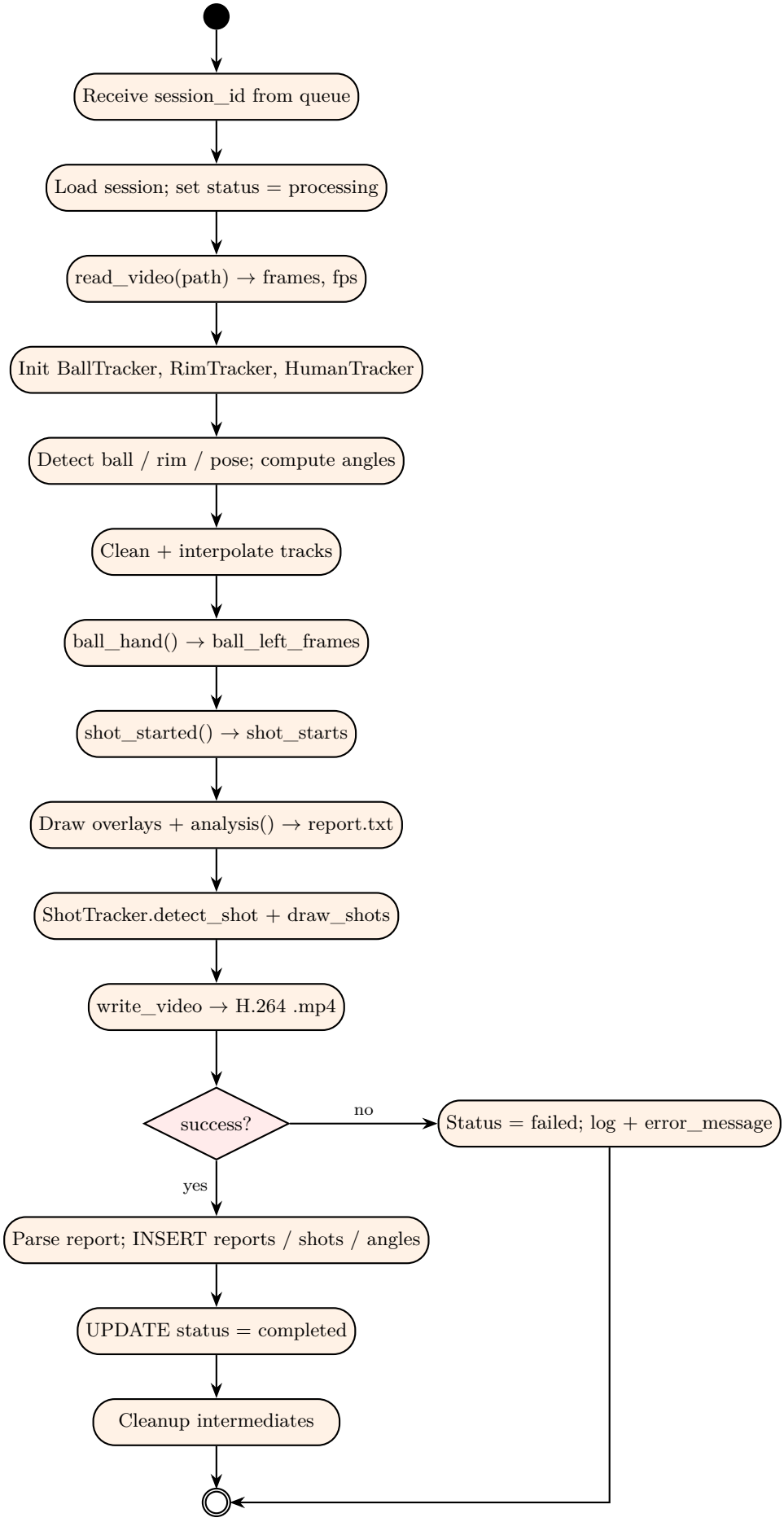
Receive session_id from queue
Load session; set status = processing
read_video(path) → frames, fps
Init BallTracker, RimTracker, HumanTracker
Detect ball / rim / pose; compute angles
Clean + interpolate tracks
ball_hand() → ball_left_frames
shot_started() → shot_starts
Draw overlays + analysis() → report.txt
ShotTracker.detect_shot + draw_shots
write_video → H.264 .mp4
success?
no
Status = failed; log + error_message
yes
Parse report; INSERT reports / shots / angles
UPDATE status = completed
Cleanup intermediates

Figure 8: Pipeline Activity Diagram

User Registration

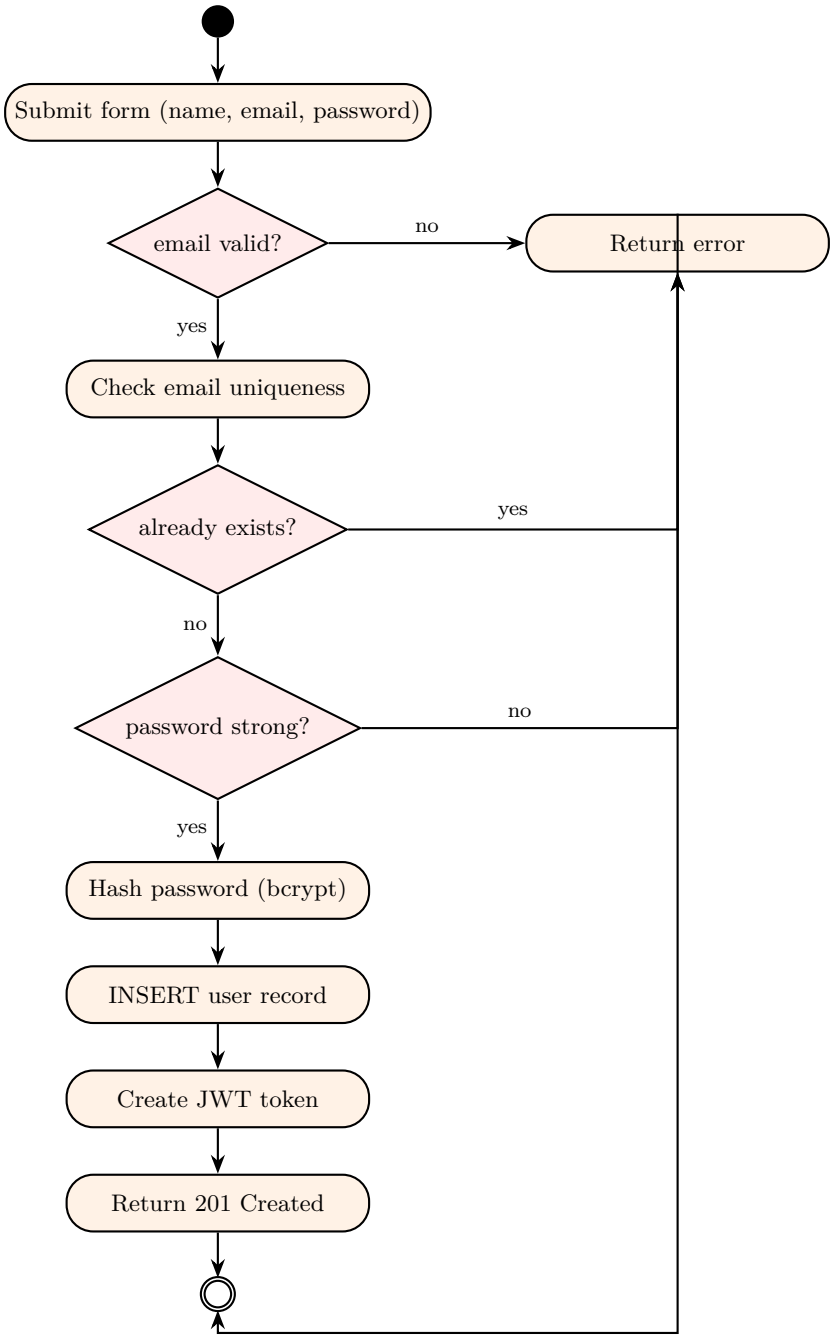

Figure 9: Registration Activity Diagram

5.2.5 System User Interface

Screen 1: Dashboard / Home

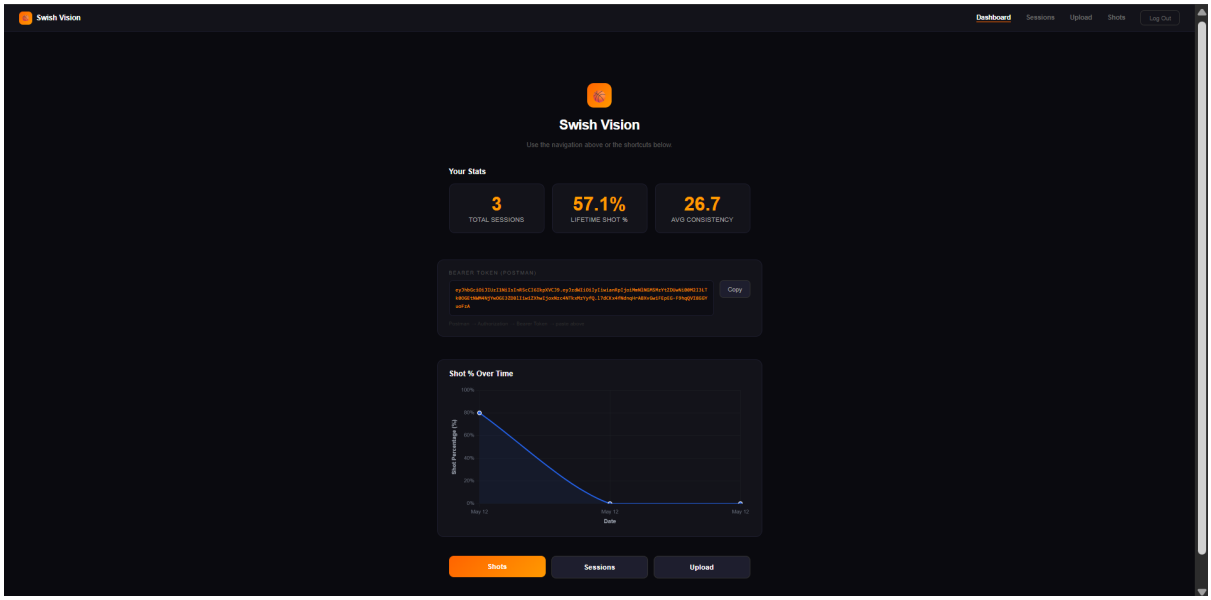

Figure 10: Dashboard

Screen 2: Upload Page

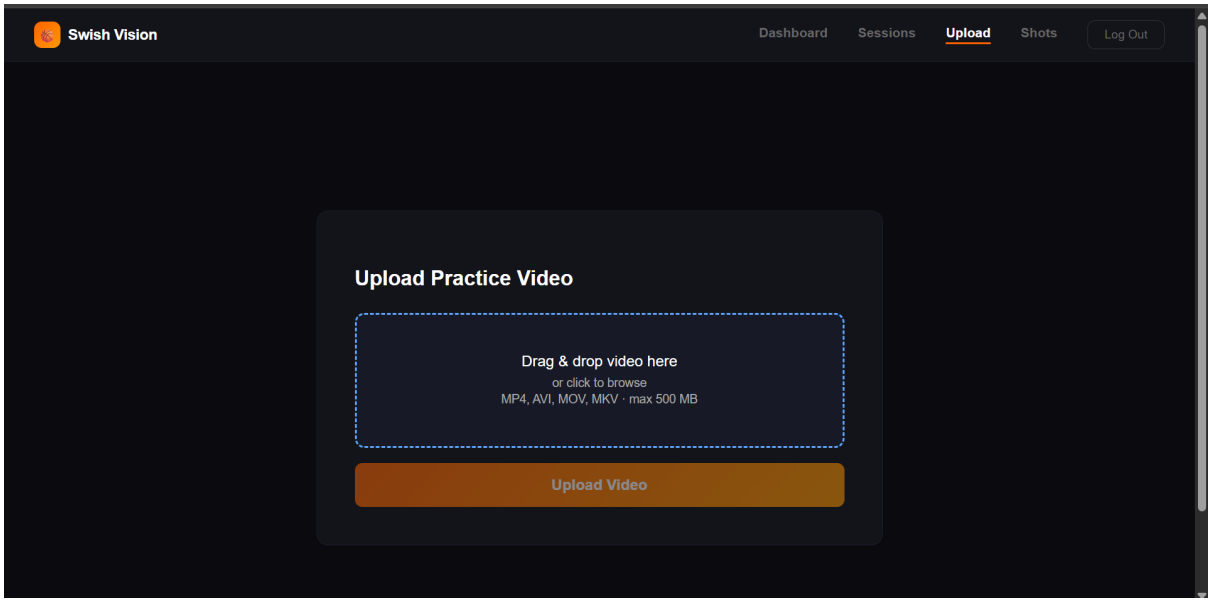

Figure 11: Upload

Screen 3: Session Results: Shot Analytics

SHOT PERCENTAGE

80.0%

4 made · 1 missed · 5 total

Made / Missed

Shot Details

#	Outcome	SEW Angle ⓘ	ESH Angle ⓘ
1	missed	87.3°	125.8°
2	made	94.0°	128.3°
3	made	86.5°	131.4°
4	made	92.0°	134.9°
5	made	85.7°	123.7°

Annotated Video

← Upload another video

Figure 12: Shot Analytics

Screen 4: Session Results: Form Analysis

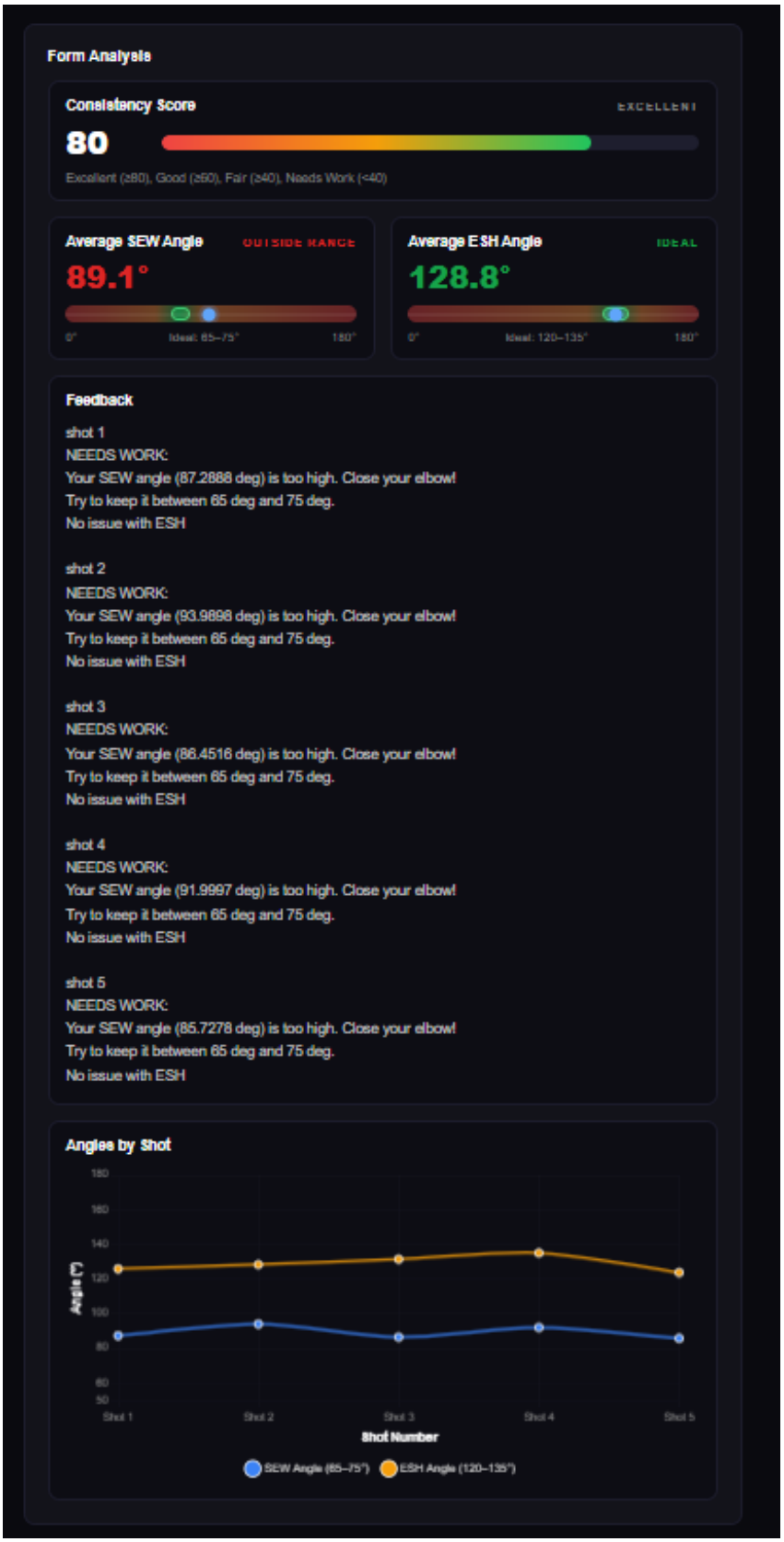

Figure 13: Form Analysis

Screen 5: Session History

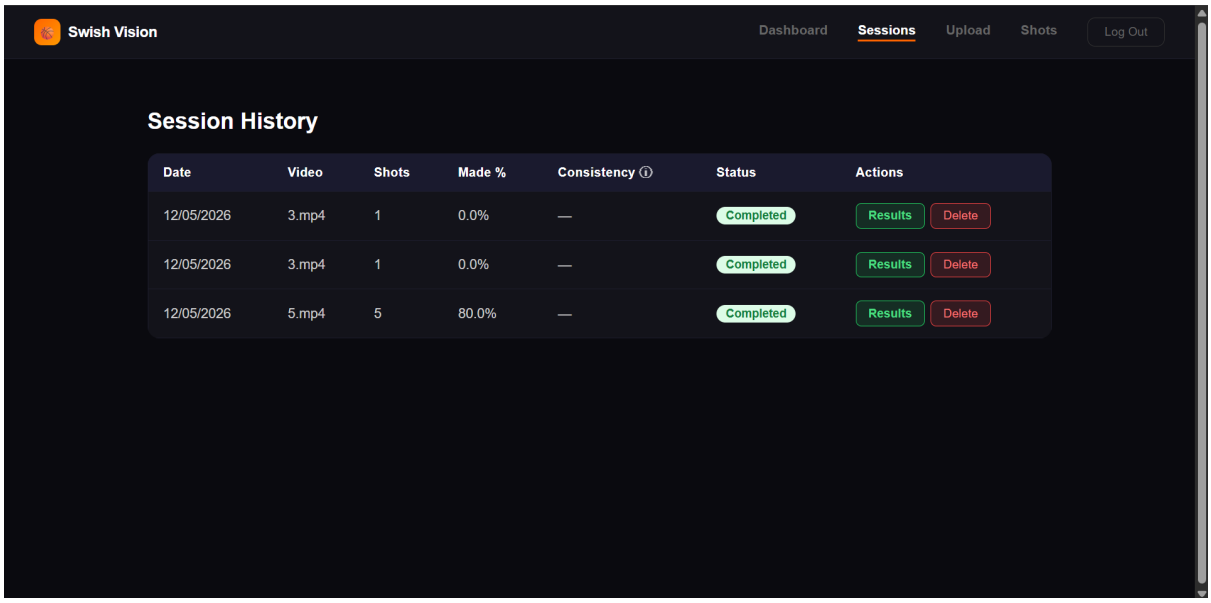

Figure 14: Session History

6 References

- Project SRS: SwishVision Software Requirements Specification v1.10
- Ultralytics YOLOv8 Documentation: `https://docs.ultralytics.com`
- Existing codebase: `main.py`, `trackers/`, `drawers/`, `utils/`

7 Appendices

Appendix A: API Endpoint Summary

Method	Endpoint	Auth	Description
POST	`/api/register`	No	Create new user account
POST	`/api/login`	No	Authenticate and receive token
POST	`/api/logout`	Yes	Invalidate current session
POST	`/api/sessions/upload`	Yes	Upload a video for processing
GET	`/api/sessions`	Yes	List sessions for user
GET	`/api/sessions/{id}`	Yes	Get session status / metadata

DELETE	/api/sessions/{id}	Yes	Delete a session and its data
GET	/api/sessions/{id}/report	Yes	Get full report data
GET	/api/sessions/{id}/shots	Yes	Get all shot events
GET	/api/sessions/{id}/output_video	Yes	Stream annotated video
GET	/api/dashboard/summary	Yes	Aggregate stats for user
GET	/api/dashboard/trends	Yes	Historical trend data

Appendix B: File System Layout

```
Swish-Vision/ # Project root
|-- main.py # Pipeline entry point
|-- trackers/ # BallTracker, RimTracker, HumanTracker
|-- drawers/ # Drawer modules + ShotTracker
|-- utils/ # vid_utils.py, ball_hand.py, stubs_utils.py
|-- models/ # best.pt, yolov8m-pose.pt
|-- input_videos/ # Worker copies uploaded video here
|-- output_videos/ # Pipeline writes H.264 .mp4 output here
|-- reports/ # Pipeline writes {vidname}_report.txt
|-- Basketball-1/ # Training dataset
|-- runs/ # YOLO training run history
|-- app/ # Web application layer (delivered)
    |-- main.py # FastAPI app factory + lifespan
    |-- config.py # Pydantic Settings (.env)
    |-- database.py # SQLAlchemy engine + SessionLocal
    |-- api/
    | |-- auth.py
    | |-- sessions.py
    | |-- dashboard.py
    |-- core/
    | |-- security.py # bcrypt + JWT helpers
    | |-- middleware.py # CORS, security headers
    |-- models/ # SQLAlchemy ORM models
    |-- schemas/ # Pydantic request/response schemas
    |-- tasks/
    | |-- pipeline_task.py # Celery app + process_video task
    | |-- report_parser.py
    |-- migrations/ # Alembic
    |-- frontend/ # React (Create React App)
    |-- uploads/ # Raw user-uploaded videos
    |-- requirements.txt
    |-- DEVELOPER_README.md
```

Appendix C: Environment Variables

Variable	Description
ENV	Environment name: development / staging / production
DB_SERVER	SQL Server hostname/instance
DB_NAME	SQL Server database name
DB_DRIVER	ODBC driver name (e.g. ODBC Driver 17 for SQL Server)
DB_TRUSTED_CONNECTION	true for Windows auth; else use DB_USERNAME/PASSWORD
DB_USERNAME / DB_PASSWORD	SQL auth credentials
REDIS_URL	Redis connection string for Celery broker
CELERY_RESULT_BACKEND	Celery result backend URL
SECRET_KEY	JWT signing secret
ADMIN_RESET_KEY	Shared secret for admin force-reset endpoint
UPLOAD_DIR	Path for raw uploaded videos
OUTPUT_DIR	Path for annotated H.264 .mp4 outputs
MODEL_DIR	Path to YOLOv8 model weights
MAX_UPLOAD_SIZE_MB	Maximum allowed upload size (default 500)